CSE310: Compiler Sessional, January 2026 Term

ICG Online Assignment (C2)

Implementing Bitwise Operators

Time: 40 Mins Total Marks: 10

Problem

In this online, you are required to extend the grammar to support **bitwise operators AND (&), OR (|), and Not (~)** maintaining their associativity and precedence as in C programming language. The operands of the bitwise operator(s) can be any expression. Thus, your program should support the following syntax.

```
int a, b;
a = 7 | 2 * b++ + 4;
a = 9 & 3 + -b - ~5;
a = 1 | 2 & 4 | 8 * 3 & ~-256;
```

Task

1. Extend the current grammar to support **bitwise operators AND (&), OR (|) and Not (~)**. You need to maintain associativity and precedence of the operators as specified below.
    a. Bitwise AND, and OR are left-associative, while bitwise NOT is right-associative.
    b. The bitwise operators have higher precedence than assignment operators but lower precedence than logical operators.
    c. The order of precedence among the operators is- NOT (~), AND (&) and OR (|) in order of decreasing precedence from left to right, i.e., NOT (~) has the highest and OR (|) has the lowest precedence among these three operators.

    There will be no syntax or semantic errors in the given input program. Additionally, in the input program, there will be no other statement in the same line containing a goto label.

2. Update your ICG code to generate assembly code for this new rule/syntax. See the sample input-output for the assembler commands that correspond to the operators. The assembly code output by your program must correctly be assembled with 'fasm' . Subsequently, the binary produced by 'fasm' from your assembly code must produce the expected output for the input program.

Mark Distribution

- Extending the grammar to support goto statement. (5)
- Code generation for goto statement. (5)

Find a sample input-output and some helper code in the next pages.

Sample Input-Output

Input Program

```
1 int a;
2 int main()
3 {
4       int b;
5       a=1;
6       b=2;
7       a = a | b;
8       println(a);
9       a = a & b;
10      println(a);
11      a = ~b;
12      println(a);
13      a= 7 | 2 * b++ + 4;
14      println(a);
15      a= 9 & 3 + -b - 5;
16      println(a);
17      a= 1 | 2 & 4 | 8 * 4 & ~-10;
18      println(a);
19      return 0;
20 }
```

Expected Output (from the generated binary by fasm)

```
3
2
-3
15
9
1
```

Generated Sample Assembly (print library omitted)

```
format ELF executable 3
entry main
segment readable writeable
        a dd 1 DUP (0)
segment readable executable
main:
        PUSH EBP
        MOV EBP, ESP
        SUB ESP, 4
.L1:
        MOV EAX, 1          ; Line 5
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L2:
        MOV EAX, 2          ; Line 6
        MOV [EBP-4], EAX
        PUSH EAX
        POP EAX
.L3:
        MOV EAX, [EBP-4]          ; Line 7
        MOV EDX, EAX
        MOV EAX, [a]          ; Line 7
        OR EAX, EDX
        PUSH EAX
        POP EAX         ; Line 7
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L4:
        MOV EAX, [a]          ; Line 8
        CALL print_number
.L5:
        MOV EAX, [EBP-4]          ; Line 9
        MOV EDX, EAX
        MOV EAX, [a]          ; Line 9
        AND EAX, EDX
        PUSH EAX
        POP EAX         ; Line 9
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L6:
        MOV EAX, [a]          ; Line 10
        CALL print_number
.L7:
        MOV EAX, [EBP-4]          ; Line 11
        NOT EAX
        PUSH EAX
        POP EAX         ; Line 11
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L8:
        MOV EAX, [a]          ; Line 12
```

```
        CALL print_number
.L9:
        MOV EAX, [EBP-4]       ; Line 13
        PUSH EAX
        INC EAX
        MOV [EBP-4], EAX
        POP EAX        ; Line 13
        MOV ECX, EAX
        MOV EAX, 2        ; Line 13
        CWD
        IMUL ECX
        PUSH EAX
        MOV EAX, 4        ; Line 13
        MOV EDX, EAX
        POP EAX        ; Line 13
        ADD EAX, EDX
        PUSH EAX
        POP EAX        ; Line 13
        MOV EDX, EAX
        MOV EAX, 7        ; Line 13
        OR EAX, EDX
        PUSH EAX
        POP EAX        ; Line 13
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L10:
        MOV EAX, [a]        ; Line 14
        CALL print_number
.L11:
        MOV EAX, [EBP-4]       ; Line 15
        NEG EAX
        PUSH EAX
        POP EAX        ; Line 15
        MOV EDX, EAX
        MOV EAX, 3        ; Line 15
        ADD EAX, EDX
        PUSH EAX
        MOV EAX, 5        ; Line 15
        MOV EDX, EAX
        POP EAX        ; Line 15
        SUB EAX, EDX
        PUSH EAX
        POP EAX        ; Line 15
        MOV EDX, EAX
        MOV EAX, 9        ; Line 15
        AND EAX, EDX
        PUSH EAX
        POP EAX        ; Line 15
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L12:
        MOV EAX, [a]        ; Line 16
        CALL print_number
.L13:
        MOV EAX, 4        ; Line 17
        MOV EDX, EAX
        MOV EAX, 2        ; Line 17
        AND EAX, EDX
```

```
        PUSH EAX
        POP EAX         ; Line 17
        MOV EDX, EAX
        MOV EAX, 1        ; Line 17
        OR EAX, EDX
        PUSH EAX
        MOV EAX, 4        ; Line 17
        MOV ECX, EAX
        MOV EAX, 8        ; Line 17
        CWD
        IMUL ECX
        PUSH EAX
        MOV EAX, 10        ; Line 17
        NEG EAX
        PUSH EAX
        POP EAX         ; Line 17
        NOT EAX
        PUSH EAX
        POP EAX         ; Line 17
        MOV EDX, EAX
        POP EAX         ; Line 17
        AND EAX, EDX
        PUSH EAX
        POP EAX         ; Line 17
        MOV EDX, EAX
        POP EAX         ; Line 17
        OR EAX, EDX
        PUSH EAX
        POP EAX         ; Line 17
        MOV [a], EAX
        PUSH EAX
        POP EAX
.L14:
        MOV EAX, [a]        ; Line 18
        CALL print_number
.L15:
        MOV EAX, 0        ; Line 19
        JMP .L17
.L16:
.L17:
        ADD ESP, 4
        POP EBP
        MOV EAX,1
        XOR EBX, EBX
        INT 0x80
        POP EBP
        RET
```